COMPUTER ENGINEERING

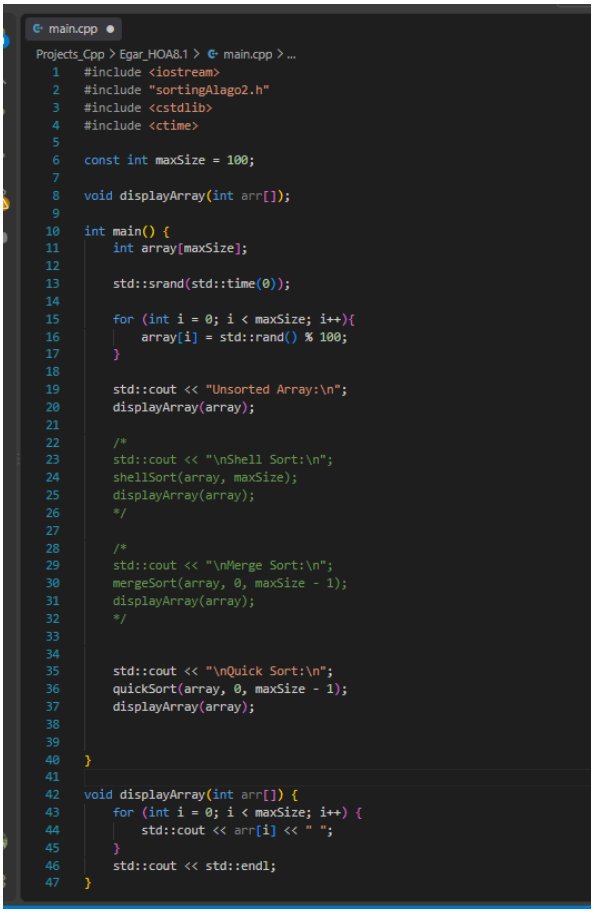
QUEZON CITY

COURSE CODE/TITLE:	CPE010 DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	09/10/2026	**DATE SUBMITTED:**	09//2026
NAME:	Jarel Dave R. Egar	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

Screenshot | Main.cpp



```cpp
#include <iostream>
#include "sortingAlago2.h"
#include <cstdlib>
#include <ctime>

const int maxSize = 100;

void displayArray(int arr[]);

int main() {
    int array[maxSize];

    std::srand(std::time(0));

    for (int i = 0; i < maxSize; i++){
        array[i] = std::rand() % 100;
    }

    std::cout << "Unsorted Array:\n";
    displayArray(array);

    /*
    std::cout << "\nShell Sort:\n";
    shellSort(array, maxSize);
    displayArray(array);
    */

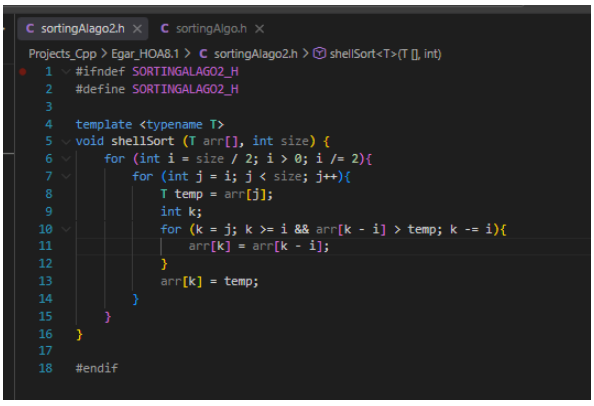
    /*
    std::cout << "\nMerge Sort:\n";
    mergeSort(array, 0, maxSize - 1);
    displayArray(array);
    */


    std::cout << "\nQuick Sort:\n";
    quickSort(array, 0, maxSize - 1);
    displayArray(array);


}

void displayArray(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

Screenshot | shellSort



```cpp
#ifndef SORTINGALAGO2_H
#define SORTINGALAGO2_H

template <typename T>
void shellSort (T arr[], int size) {
    for (int i = size / 2; i > 0; i /= 2){
        for (int j = i; j < size; j++){
            T temp = arr[j];
            int k;
            for (k = j; k >= i && arr[k - i] > temp; k -= i){
                arr[k] = arr[k - i];
            }
            arr[k] = temp;
        }
    }
}

#endif
```

Screenshot | mergeSort

```cpp
//helper function for merge sort
template <typename T>
void merge(T arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    T* L = new T[n1];
    T* R = new T[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0;
    int j = 0;
    int k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

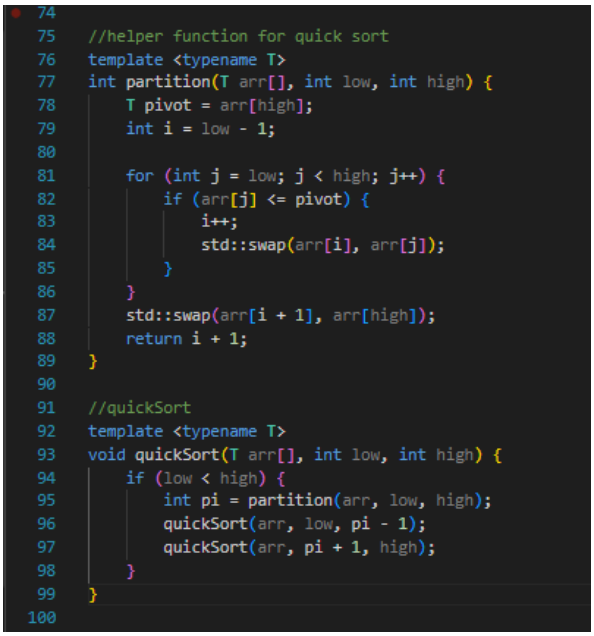
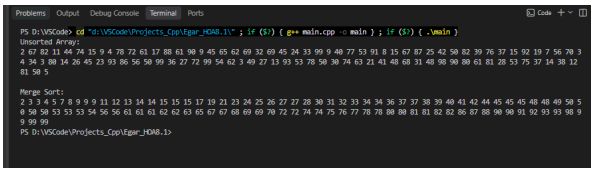
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}
```

```cpp

//Merge Sort
template <typename T>
void mergeSort (T arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

```

Screenshot | quickSort	
Output | shellSort	
Output | mergeSort	
Output | quickSort	

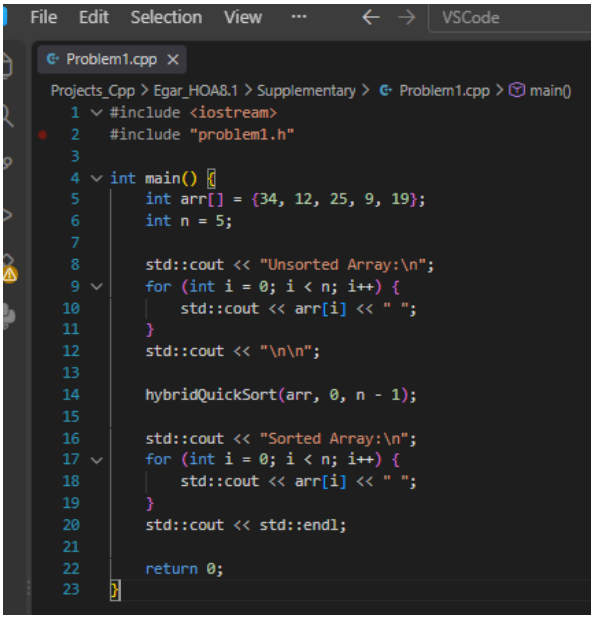
```
74
75  //helper function for quick sort
76  template <typename T>
77  int partition(T arr[], int low, int high) {
78      T pivot = arr[high];
79      int i = low - 1;
80
81      for (int j = low; j < high; j++) {
82          if (arr[j] <= pivot) {
83              i++;
84              std::swap(arr[i], arr[j]);
85          }
86      }
87      std::swap(arr[i + 1], arr[high]);
88      return i + 1;
89  }
90
91  //quickSort
92  template <typename T>
93  void quickSort(T arr[], int low, int high) {
94      if (low < high) {
95          int pi = partition(arr, low, high);
96          quickSort(arr, low, pi - 1);
97          quickSort(arr, pi + 1, high);
98      }
99  }
100
```

CODE ANALYSIS: In the header file I included functions to implement Shell Sort, Merge Sort and Quick Sort in such a way that they can be used with any data type. To reassemble the smaller sorted sections once the array has been divided in half, I wrote a merge function which is then used by mergeSort. With regard to Quick Sort, I developed a partition helper function which selects a pivot element, swaps the smaller elements to the left side and places the pivot in the right position. In main.cpp I set up an array containing 100 random numbers and printed them out in order to display the unsorted list. I then called quickSort to sort the numbers from lowest to highest and printed out the finally sorted array.

SUPPLEMENTARY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Yes, you can sort the left and the right by partition method. This means that after pivoting the lemetns it will settle to the corrected sorted index. This is an example of it:

Screenshot | Problem1.cpp	

Screenshot | Problem1.h

problem1.h

Projects_Cpp > Egar_HOA8.1 > Supplementary > problem1.h > ...

```cpp
#ifndef PROBLEM1_H
#define PROBLEM1_H

// Partition function from Quick Sort
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            T temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    T temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

// Simple Selection Sort for sub-lists
template <typename T>
void selectionSort(T arr[], int start, int end) {
    for (int i = start; i < end; i++) {
        int minIdx = i;
        for (int j = i + 1; j <= end; j++) {
            if (arr[j] < arr[minIdx]) {
                minIdx = j;
            }
        }
        T temp = arr[i];
        arr[i] = arr[minIdx];
        arr[minIdx] = temp;
    }
}

// Partitions first, then sorts left and right sub-lists
template <typename T>
void hybridQuickSort(T arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        // Sort sub-lists using selection sort
        selectionSort(arr, low, pi - 1);
        selectionSort(arr, pi + 1, high);
    }
}

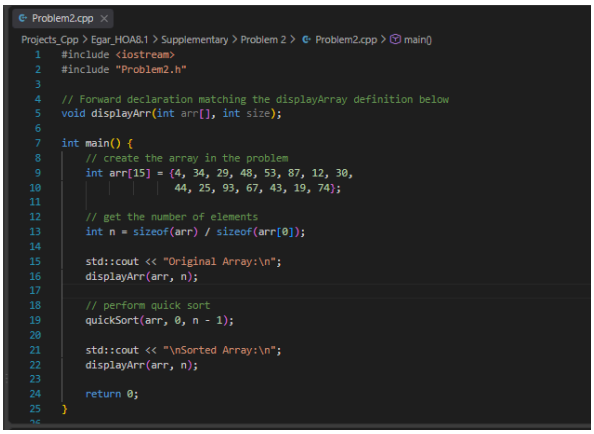
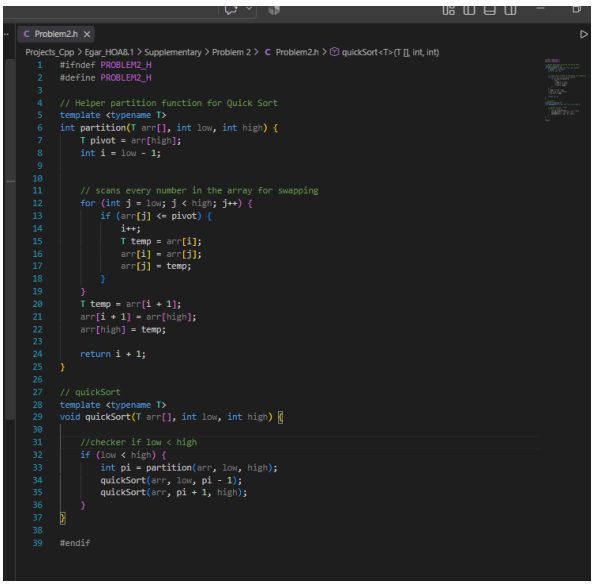
#endif
```

Output

```
Quick Sort:
0 0 2 2 3 4 4 4 4 4 6 8 9 10 10 11 11 11 13 14 14 15 16 16 16 18 18 18 23 24 24 24 26 27 28 28 29 29 31 32 32 32 34 34 34 35 40 42 42 44 45 45 46 46 48 50 52 52 55 57 58 58 59
71 71 73 76 76 77 77 78 78 83 84 84 85 85 85 86 86 86 87 89 90 91 92 94 97 97 99
Unsorted Array:
34 12 25 9 19

Sorted Array:
9 12 19 25 34
PS D:\VSCode\Projects_Cpp\Egar_HOA8.1\Supplementary>
```

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

Screenshot | Problem2.cpp	
Screenshot | Problem2.h	
Output	

Code analysis: In the header file, I made a partition helper that uses the last item as a pivot. It loops through the array to move smaller numbers to the front and puts the pivot in its right spot. Below it, I wrote quickSort to split and sort the left and right sides over and over as long as low < high.

In Problem2.cpp, I added a function preview for displayArr and created my list of 15 numbers inside main(). I used sizeof(arr) / sizeof(arr[0]) to count how many numbers were in the array. Lastly, I printed the original list, sorted it with quickSort, and printed the final sorted numbers.

CONCLUSION: This activity has helped me understand merge, shell, and quick sort. During the discussion I was shocked to hear that there where so many sorting algorithms Working on the code showed me how each algorithm splits or compares numbers in its own unique way to order a list. I learned that Quick Sort is very fast because it sorts items in place without using extra memory space. Overall, this lab made me realize how choosing the right sorting method depends on the size of the data and how fast it needs to run.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation for problem 1**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I asked Gemini to create a step-by-step process on how to code problem 1 but I asked it to not give the code directly to me but only the pseudocode**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.